rf_occupancy_train

April 27, 2026

1 Random Forest Occupancy Training (Full Pipeline)

End-to-end occupancy classifier for Room 354 FPB.

- **Target:** `occupied_schedule` (weak label from AIEB 354 enrollment schedule)
- **Features:** sensor-only — multiple rolling windows + lag features + range features. No schedule leakage.
- **Model selection:** stratified `GridSearchCV` over a real hyperparameter grid
- **Validation:** also reports `TimeSeriesSplit` CV scores so we see degradation across time
- **Diagnostics:** classification report, confusion matrix, ROC + PR curves, permutation importance

This notebook is intentionally compute-heavy. Expect several minutes of training.

[20]:
```
import re
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import (
    GridSearchCV,
    StratifiedKFold,
    TimeSeriesSplit,
    cross_val_score,
    train_test_split,
)
from sklearn.metrics import (
    PrecisionRecallDisplay,
    RocCurveDisplay,
    classification_report,
    confusion_matrix,
    f1_score,
)
from sklearn.inspection import permutation_importance
```

1.1 Step 0 — Load data and attach schedule labels

[21]:
```
PROJECT_ROOT = Path.cwd() if (Path.cwd() / 'data').exists() else Path.cwd().
 ↪parent
DATA_FILE = PROJECT_ROOT / 'data/03-28_to_04-27_rm354_FPB(cleaned).csv'
SCHEDULE_FILE = PROJECT_ROOT / 'data/AIEB 354 Enrollment Numbers.txt'
ROOM_CAPACITY = 60


def parse_time_token(token: str):
    return pd.to_datetime(token, format='%I%M%p').time()


def parse_enrollment_schedule(path: Path) -> pd.DataFrame:
    rows = []
    for raw_line in path.read_text(encoding='utf-8', errors='replace').
 ↪splitlines():
        line = raw_line.strip()
        if not line or 'AIEB 354' not in line:
            continue

        tokens = line.split()
        enrollment_idx = next((i for i, token in enumerate(tokens) if re.
 ↪fullmatch(r'\d+/\d+', token)), None)
        if enrollment_idx is None or len(tokens) < 5:
            continue

        enrolled, capacity = (int(value) for value in tokens[enrollment_idx].
 ↪split('/'))
        start_token, end_token = tokens[3].split('-')
        rows.append({
            'course': ' '.join(tokens[:2]),
            'days': tokens[4],
            'start_time': parse_time_token(start_token),
            'end_time': parse_time_token(end_token),
            'enrolled': enrolled,
            'capacity': capacity,
        })

    base = pd.DataFrame(rows)
    day_map = {'M': 0, 'T': 1, 'W': 2, 'R': 3, 'F': 4}
    exploded = []
    for row in base.itertuples(index=False):
        for day in row.days:
            expanded = row._asdict().copy()
            expanded['day_of_week'] = day_map[day]
```

```
            expanded['start_minutes'] = row.start_time.hour * 60 + row.
 ↪start_time.minute
            expanded['end_minutes'] = row.end_time.hour * 60 + row.end_time.
 ↪minute
            exploded.append(expanded)
    return pd.DataFrame(exploded).sort_values(['day_of_week', 'start_minutes',␣
 ↪'course'])


def add_schedule_context(frame: pd.DataFrame, schedule: pd.DataFrame) -> pd.
 ↪DataFrame:
    result = frame.copy()
    minute_of_day = result['time'].dt.hour * 60 + result['time'].dt.minute

    scheduled_count = []
    for ts, minute in zip(result['time'], minute_of_day):
        hits = schedule[
            (schedule['day_of_week'] == ts.dayofweek)
            & (schedule['start_minutes'] <= minute)
            & (schedule['end_minutes'] > minute)
        ]
        scheduled_count.append(int(hits['enrolled'].sum()))

    result['scheduled_count'] = np.clip(scheduled_count, 0, ROOM_CAPACITY)
    result['occupied_schedule'] = (result['scheduled_count'] > 0).astype(int)
    return result


df = pd.read_csv(DATA_FILE)
df.columns = df.columns.str.strip()
df['time'] = pd.to_datetime(df['time'])
schedule_df = parse_enrollment_schedule(SCHEDULE_FILE)
df = add_schedule_context(df, schedule_df)

print(f'Loaded {len(df):,} rows from {DATA_FILE.name}')
print(f'Time range: {df["time"].min()} to {df["time"].max()}')
```

```
Loaded 42,753 rows from 03-28_to_04-27_rm354_FPB(cleaned).csv
Time range: 2026-03-29 00:00:00 to 2026-04-27 16:32:00
```

1.2 Step 1 — Rich feature engineering

We build a wide bank of *behavioral* features. The target is schedule-derived, so we deliberately exclude any time-of-day or schedule columns — the model has to learn occupancy from sensor dynamics alone.

Per sensor (`co2`, `air_flow`, `temperature`, `humidity`): - Rolling **mean** and **std** at 5, 15, 30, and 60-minute windows - Rolling **min/max range** at 30 minutes - First difference (rate of change) -

5-min and 15-min lagged values

That gives ~60 features. Random Forest handles correlated features well and the extra signal is what makes it worth searching hyperparameters.

[22]:
```
ts = df.set_index('time').sort_index()
numeric_cols = ts.select_dtypes(include='number').columns.tolist()
ts = ts[numeric_cols].resample('1min').mean().interpolate(method='time')
ts['occupied_schedule'] = (ts['scheduled_count'] > 0).astype(int)

feat = ts.copy()
sensor_cols = ['co2', 'air_flow', 'temperature', 'humidity']
rolling_windows = [5, 15, 30, 60]
lag_steps = [5, 15]

for col in sensor_cols:
    for w in rolling_windows:
        feat[f'{col}_roll{w}_mean'] = feat[col].rolling(w).mean()
        feat[f'{col}_roll{w}_std']  = feat[col].rolling(w).std()
    feat[f'{col}_range30'] = (
        feat[col].rolling(30).max() - feat[col].rolling(30).min()
    )
    feat[f'{col}_diff'] = feat[col].diff()
    for lag in lag_steps:
        feat[f'{col}_lag{lag}'] = feat[col].shift(lag)

# Stronger behavioral features
feat['co2_acceleration'] = feat['co2_diff'].diff()
feat['airflow_ratio'] = feat['air_flow'] / feat['air_flow_roll30_mean'].
 ↪replace(0, np.nan)
feat['airflow_ratio'] = feat['airflow_ratio'].replace([np.inf, -np.inf], np.nan)

feat = feat.dropna()

rf_features = [
    c for c in feat.columns
    if c not in {'occupied_schedule', 'scheduled_count'}
    and not c.startswith('expected')
    and not c.startswith('active')
    and not c.startswith('is_class')
]
print(f'Rows after feature engineering: {len(feat):,}')
print(f'Feature count: {len(rf_features)}')
print(rf_features[:10], '...')
```

```
Rows after feature engineering: 42,694
Feature count: 54
['humidity', 'air_flow', 'co2', 'temperature', 'co2_roll5_mean',
```

```
'co2_roll5_std', 'co2_roll15_mean', 'co2_roll15_std', 'co2_roll30_mean',
'co2_roll30_std'] …
```

1.3 Step 2 — Define target and time-based train/test split

A random split lets the model peek at minutes adjacent to those it's predicting, which is why the random-split F1 (~0.97) and the `TimeSeriesSplit` F1 (~0.19) disagree so violently. Switching to a chronological split forces the realistic question: *given past sensor history, can we predict occupancy for unseen future days?*

[23]:
```
X = feat[rf_features]
y = feat['occupied_schedule']

print('Class balance:')
print(y.value_counts().rename(index={0: 'unoccupied', 1: 'occupied'}))
print(f'Positive rate: {y.mean():.3f}')

# Time-based split: train on past, predict future
SPLIT_TIME = pd.Timestamp('2026-04-20')
train_mask = feat.index < SPLIT_TIME
test_mask  = feat.index >= SPLIT_TIME

X_train, X_test = X.loc[train_mask], X.loc[test_mask]
y_train, y_test = y.loc[train_mask], y.loc[test_mask]

print(f'\nSplit at {SPLIT_TIME.date()}')
print(f'Train: {len(X_train):,}   ({y_train.mean():.3f} positive)   {X_train.
  ↪index.min()} → {X_train.index.max()}')
print(f'Test:  {len(X_test):,}   ({y_test.mean():.3f} positive)   {X_test.index.
  ↪min()} → {X_test.index.max()}')
```

```
Class balance:
occupied_schedule
unoccupied    38286
occupied       4408
Name: count, dtype: int64
Positive rate: 0.103

Split at 2026-04-20
Train: 31,621   (0.095 positive)   2026-03-29 00:59:00 → 2026-04-19 23:59:00
Test:  11,073   (0.127 positive)   2026-04-20 00:00:00 → 2026-04-27 16:32:00
```

1.4 Step 3 — Hyperparameter grid search (regularized)

Two changes vs. the previous run:

1. **CV uses `TimeSeriesSplit`** instead of `StratifiedKFold` — this is what we actually care about generalizing to, so we should select hyperparameters against it.

2. **Grid favors regularization.** The previous best model used `max_depth=None` (unlimited), which is exactly what overfits in a temporal setting. We cap depth and increase `min_samples_leaf` so trees can't memorize per-minute noise.

Scoring is still F1 of the occupied class because the data is imbalanced (~10% positive).

[24]:
```
param_grid = {
    'n_estimators':     [300, 600, 900],
    'max_depth':        [6, 10, 14],
    'min_samples_leaf': [10, 20, 40],
    'max_features':     ['sqrt', 'log2'],
}
n_combos = (
    len(param_grid['n_estimators'])
    * len(param_grid['max_depth'])
    * len(param_grid['min_samples_leaf'])
    * len(param_grid['max_features'])
)
n_splits = 5
print(f'Searching {n_combos} hyperparameter combinations × {n_splits} folds =␣
 ↪{n_combos * n_splits} fits')

base_rf = RandomForestClassifier(
    class_weight='balanced',
    random_state=42,
    n_jobs=-1,
)

# Time-based CV so hyperparameters are selected for forward generalization
cv = TimeSeriesSplit(n_splits=n_splits)
search = GridSearchCV(
    estimator=base_rf,
    param_grid=param_grid,
    scoring='f1',
    cv=cv,
    n_jobs=-1,
    verbose=2,
    refit=True,
)

t0 = time.perf_counter()
search.fit(X_train, y_train)
elapsed = time.perf_counter() - t0
print(f'\nGrid search completed in {elapsed/60:.1f} min')
print(f'Best CV F1 (TimeSeriesSplit): {search.best_score_:.4f}')
print(f'Best params: {search.best_params_}')
```

```
Searching 54 hyperparameter combinations × 5 folds = 270 fits
```

```
Fitting 5 folds for each of 54 candidates, totalling 270 fits
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   2.5s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   3.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   3.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=   3.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   4.9s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   6.2s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=   6.1s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=   7.7s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=   9.9s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  14.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  13.1s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  14.7s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  14.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   6.0s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  11.5s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  13.4s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   7.4s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  18.6s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  19.8s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  16.2s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  20.1s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  15.6s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   8.9s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
```

```
total time=  10.2s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=   9.5s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  21.7s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  26.0s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  23.5s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  23.9s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  25.5s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   6.2s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=  13.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  16.4s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  12.5s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   6.0s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  15.9s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   7.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  20.9s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  14.8s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  29.3s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   8.1s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   8.8s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=   8.6s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  16.0s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  20.4s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  10.5s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=   8.3s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=300;
```

```
total time=   8.5s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  37.5s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=  10.0s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  16.3s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  26.9s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  25.9s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  10.8s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=  11.6s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  36.8s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=  12.5s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  17.0s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  19.2s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  20.0s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  14.1s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  19.6s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  30.6s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   5.3s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   5.9s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  15.9s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   6.4s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  21.6s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  12.3s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   8.6s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   9.8s
[CV] END max_depth=6, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
```

```
total time=  36.2s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=   9.8s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  18.8s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  15.0s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  19.8s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  26.4s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  10.9s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   5.7s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  12.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   6.9s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  17.8s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  11.6s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  30.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   8.9s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  15.7s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  15.0s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  22.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=   5.7s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  17.3s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  10.4s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=  12.9s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  19.2s[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20,
n_estimators=300; total time=   7.2s

[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  11.7s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=20, n_estimators=900;
```

```
total time=  25.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=  18.4s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  18.4s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  13.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  19.7s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  27.9s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  14.1s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   5.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  17.6s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  17.0s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  24.1s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  26.5s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  20.3s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   7.6s
[CV] END max_depth=6, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  34.0s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  18.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   9.9s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  21.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  10.5s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  31.1s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  22.6s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=  13.5s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  33.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  13.4s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=300;
```

```
total time=   6.1s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=  16.2s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   7.3s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  29.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  17.6s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  31.0s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   9.2s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  15.0s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  40.5s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  34.1s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  10.3s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=  11.3s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  10.8s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=  17.1s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  23.9s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=   8.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  26.1s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  50.2s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=   9.9s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  18.6s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  43.3s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=  10.8s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  17.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  29.5s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
```

```
total time=  32.6s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=   8.8s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=  13.6s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  23.1s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  20.7s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=  11.9s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  45.3s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   5.7s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  11.4s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  25.8s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   6.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  36.5s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  12.7s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  24.1s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   7.6s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  15.1s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=   9.2s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=  10.0s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=   9.6s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  17.9s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=  11.1s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  18.2s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   5.2s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  32.0s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
```

```
total time=   7.9s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  45.4s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  23.5s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  13.9s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  21.0s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  13.6s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=   9.5s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  18.6s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  40.6s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  21.5s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  14.1s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=   8.5s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  30.6s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=  16.3s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  10.1s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  16.7s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  24.4s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=300;
total time=  20.1s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  27.8s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  13.3s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  17.3s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  38.2s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  22.7s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  10.2s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
```

```
total time=  13.3s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   6.6s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  23.8s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  33.1s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  14.2s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  12.0s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=   8.5s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=300;
total time=  17.4s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  29.8s
[CV] END max_depth=10, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  36.0s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=  11.6s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  20.7s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  32.1s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  11.1s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  20.4s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=  14.1s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=600;
total time=  36.5s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  13.6s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=300;
total time=  16.9s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=   8.6s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  29.7s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  19.2s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=  10.8s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  18.7s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
```

```
total time=  31.0s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=  13.2s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  45.5s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  12.7s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=  14.4s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=600;
total time=  36.8s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  24.2s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=300;
total time=  16.9s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  15.6s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=   4.8s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  29.3s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=   7.1s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  13.4s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=   8.6s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  19.8s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  41.3s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  31.0s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=10, n_estimators=900;
total time=  55.7s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  15.1s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=   8.0s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=300;
total time=  11.3s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=600;
total time=  35.2s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  22.4s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   5.6s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=300;
```

```
total time=  15.1s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  13.9s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   6.9s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=   8.9s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=600;
total time=  26.9s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=20, n_estimators=900;
total time=  52.2s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  17.5s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=  10.2s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  25.4s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  14.2s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  42.3s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  13.1s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  18.6s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=300;
total time=  14.8s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  23.8s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  32.8s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  13.9s
[CV] END max_depth=14, max_features=sqrt, min_samples_leaf=40, n_estimators=900;
total time=  50.8s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  22.7s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  15.7s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=600;
total time=  26.0s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  15.1s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=10, n_estimators=900;
total time=  35.9s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  17.8s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=900;
```

```
total time=  15.2s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=600;
total time=  18.4s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  26.3s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=20, n_estimators=900;
total time=  27.2s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  16.5s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  16.4s
[CV] END max_depth=14, max_features=log2, min_samples_leaf=40, n_estimators=900;
total time=  16.7s

Grid search completed in 3.5 min
Best CV F1 (TimeSeriesSplit): 0.2230
Best params: {'max_depth': 14, 'max_features': 'sqrt', 'min_samples_leaf': 40,
'n_estimators': 300}
```

[25]:
```
cv_results = pd.DataFrame(search.cv_results_)
top = (
    cv_results[['mean_test_score', 'std_test_score', 'mean_fit_time', 'params']]
    .sort_values('mean_test_score', ascending=False)
    .head(10)
    .reset_index(drop=True)
)
top.round(4)
```

[25]:

	mean_test_score	std_test_score	mean_fit_time	params
0	0.2230	0.1710	10.5820	{'max_depth': 14, 'max_features': 'sqrt', 'min…
1	0.2214	0.1814	8.4009	{'max_depth': 6, 'max_features': 'sqrt', 'min_…
2	0.2206	0.1819	23.7010	{'max_depth': 6, 'max_features': 'sqrt', 'min_…
3	0.2205	0.1426	11.8547	{'max_depth': 14, 'max_features': 'sqrt', 'min…
4	0.2191	0.1346	11.1046	{'max_depth': 14, 'max_features': 'sqrt', 'min…
5	0.2190	0.1412	30.1562	{'max_depth': 14, 'max_features': 'sqrt', 'min…
6	0.2183	0.1618	9.9553	
7	0.2181	0.1423	21.1508	
8	0.2169	0.1795	16.1343	
9	0.2162	0.1329	33.7817	

```
6  {'max_depth': 10, 'max_features': 'sqrt', 'min…
7  {'max_depth': 14, 'max_features': 'sqrt', 'min…
8  {'max_depth': 6, 'max_features': 'sqrt', 'min_…
9  {'max_depth': 14, 'max_features': 'sqrt', 'min…
```

1.5 Step 4 — Time-series cross-validation sanity check

Stratified CV measures how well the model generalizes to held-out *minutes*. For deployment, we also want to know how well it generalizes to held-out *days*. `TimeSeriesSplit` trains on earlier data and tests on later data — a stricter test.

[26]:
```
best_model = search.best_estimator_

tscv = TimeSeriesSplit(n_splits=5)
tscv_scores = cross_val_score(
    best_model, X, y, cv=tscv, scoring='f1', n_jobs=-1,
)
print('TimeSeriesSplit F1 per fold:', np.round(tscv_scores, 4))
print(f'Mean: {tscv_scores.mean():.4f}   Std: {tscv_scores.std():.4f}')
```

```
TimeSeriesSplit F1 per fold: [0.1146 0.2026 0.3953 0.4295 0.2813]
Mean: 0.2847   Std: 0.1174
```

1.6 Step 5 — Hold-out evaluation

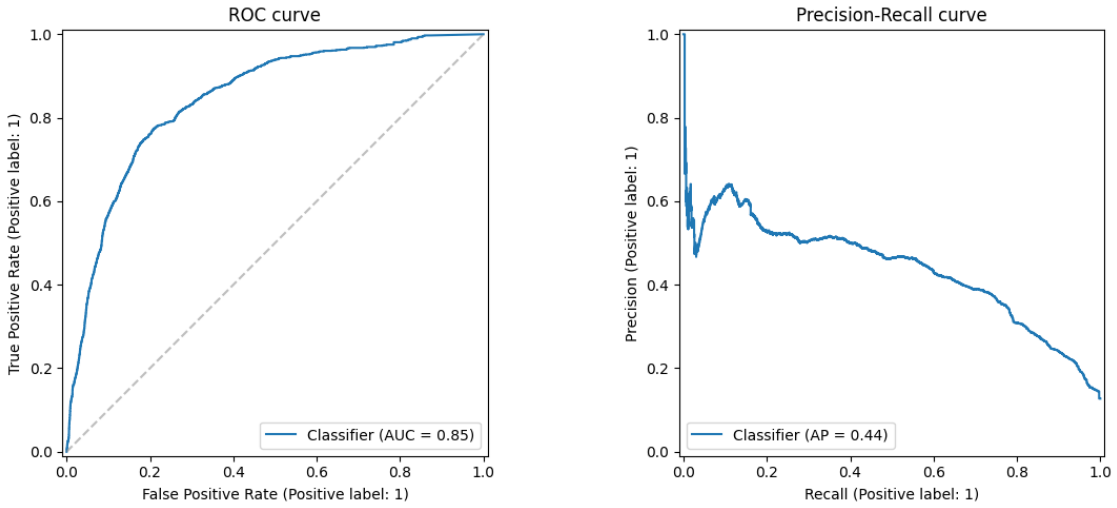
Final report on the 30% test set the model has not seen during search.

[27]:
```
y_pred  = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred, target_names=['unoccupied',␣
 ↪'occupied']))
print('Confusion matrix:')
print(confusion_matrix(y_test, y_pred))
print(f'\nHold-out F1 (occupied): {f1_score(y_test, y_pred):.4f}')
```

```
              precision    recall  f1-score   support

  unoccupied       0.91      0.94      0.93      9665
    occupied       0.50      0.39      0.44      1408

    accuracy                           0.87     11073
   macro avg       0.71      0.67      0.69     11073
weighted avg       0.86      0.87      0.87     11073

Confusion matrix:
[[9122  543]
 [ 855  553]]
```

```
Hold-out F1 (occupied): 0.4417
```

[28]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
RocCurveDisplay.from_predictions(y_test, y_proba, ax=axes[0])
axes[0].set_title('ROC curve')
axes[0].plot([0, 1], [0, 1], '--', color='gray', alpha=0.5)
PrecisionRecallDisplay.from_predictions(y_test, y_proba, ax=axes[1])
axes[1].set_title('Precision-Recall curve')
plt.tight_layout()
plt.show()
```



1.7 Step 6 — Permutation feature importance

Slower but more honest than the default impurity-based importance: each feature is shuffled in the test set and we measure how much F1 drops. Highly correlated features will share importance instead of one stealing all of it.

This cell is also intentionally slow — 10 repeats × ~60 features.

[29]:
```
t0 = time.perf_counter()
perm = permutation_importance(
    best_model, X_test, y_test,
    n_repeats=10,
    random_state=42,
    scoring='f1',
    n_jobs=-1,
)
print(f'Permutation importance computed in {time.perf_counter() - t0:.1f}s')

perm_df = (
```

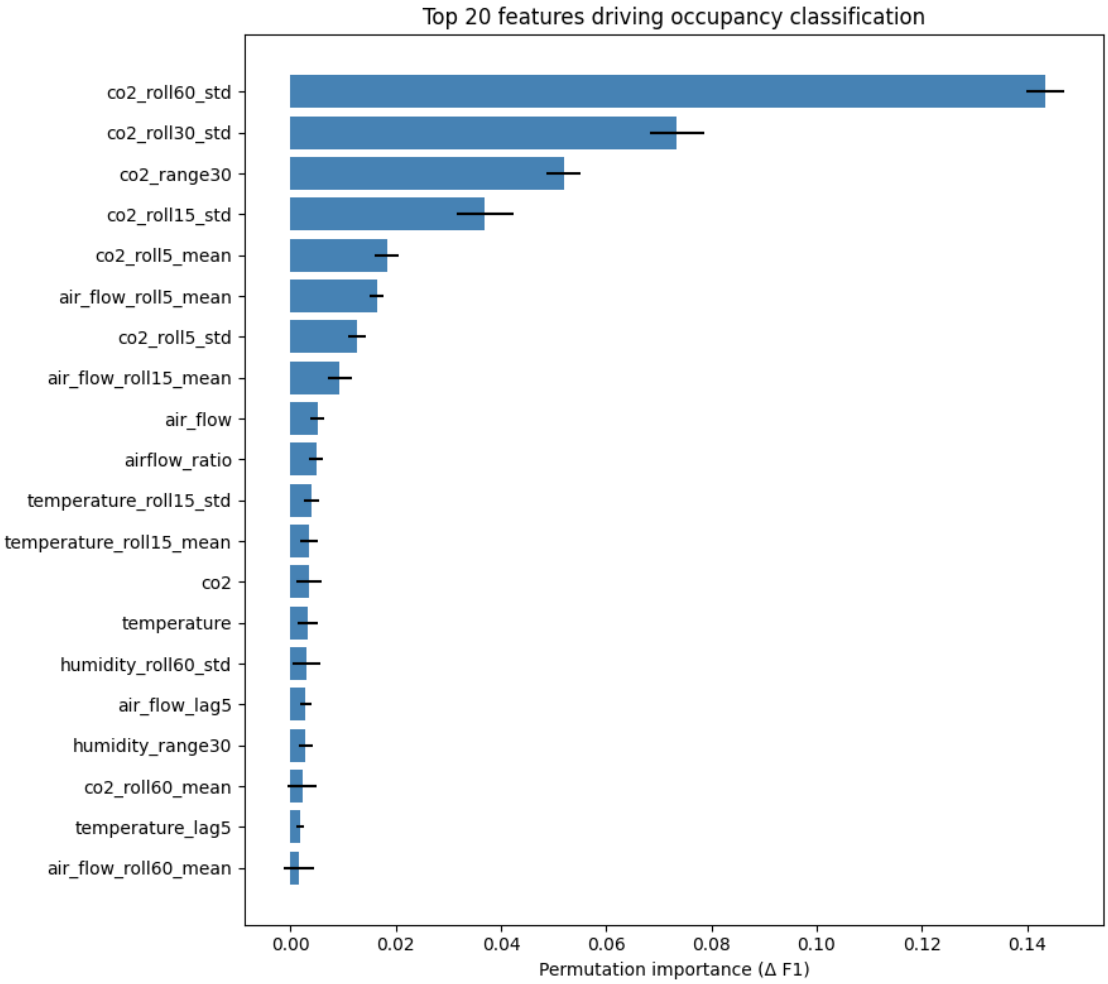
```
    pd.DataFrame({
        'feature':    rf_features,
        'importance': perm.importances_mean,
        'std':        perm.importances_std,
    })
    .sort_values('importance', ascending=False)
    .reset_index(drop=True)
)
perm_df.head(20).round(4)
```

```
Permutation importance computed in 6.8s
```

[29]:

	feature	importance	std
0	co2_roll60_std	0.1434	0.0036
1	co2_roll30_std	0.0734	0.0052
2	co2_range30	0.0519	0.0032
3	co2_roll15_std	0.0370	0.0054
4	co2_roll5_mean	0.0183	0.0022
5	air_flow_roll5_mean	0.0164	0.0012
6	co2_roll5_std	0.0127	0.0018
7	air_flow_roll15_mean	0.0093	0.0023
8	air_flow	0.0052	0.0013
9	airflow_ratio	0.0049	0.0013
10	temperature_roll15_std	0.0041	0.0014
11	temperature_roll15_mean	0.0036	0.0017
12	co2	0.0035	0.0024
13	temperature	0.0033	0.0020
14	humidity_roll60_std	0.0031	0.0026
15	air_flow_lag5	0.0028	0.0011
16	humidity_range30	0.0028	0.0013
17	co2_roll60_mean	0.0022	0.0027
18	temperature_lag5	0.0019	0.0008
19	air_flow_roll60_mean	0.0016	0.0029

[30]:
```
top20 = perm_df.head(20).iloc[::-1]
fig, ax = plt.subplots(figsize=(9, 8))
ax.barh(top20['feature'], top20['importance'], xerr=top20['std'],␣
 ↪color='steelblue')
ax.set_xlabel('Permutation importance (Δ F1)')
ax.set_title('Top 20 features driving occupancy classification')
plt.tight_layout()
plt.show()
```


Top 20 features driving occupancy classification
co2_roll60_std
co2_roll30_std
co2_range30
co2_roll15_std
co2_roll5_mean
air_flow_roll5_mean
co2_roll5_std
air_flow_roll15_mean
air_flow
airflow_ratio
temperature_roll15_std
temperature_roll15_mean
co2
temperature
humidity_roll60_std
air_flow_lag5
humidity_range30
co2_roll60_mean
temperature_lag5
air_flow_roll60_mean
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
Permutation importance (Δ F1)